

Spring is a core Java framework used for developing enterprise applications. It provides features like Dependency Injection, IoC, AOP, and transaction management.

Spring MVC is a module of Spring used specifically for developing web applications and REST APIs. It follows the Model View Controller architecture and handles HTTP requests using controllers.

Spring Boot is an extension of Spring that simplifies application development by reducing configuration. It provides features like auto-configuration, starter dependencies, and embedded servers such as Tomcat, which helps us develop and deploy applications faster.”

“Spring Container is the core part of the Spring Framework that creates, manages, and injects objects called beans.

It is responsible for:

- Creating objects
- Managing bean lifecycle
- Injecting dependencies
- Configuring the application

Examples of Spring containers are:

- BeanFactory
- ApplicationContext”

“Dependency Injection is a process where the required dependency objects are provided to a class by the Spring container instead of creating them manually using the new keyword. It helps in achieving loose coupling and makes the application easier to maintain and test.”

“IoC (Inversion of Control) is a principle in which the control of object creation and dependency handling is given to the Spring container instead of the programmer. The Spring container manages the objects and injects the required dependencies automatically.”

“There are mainly two types of Dependency Injection in Spring:

1. Constructor Injection
→ Dependencies are injected through the constructor.
2. Setter Injection
→ Dependencies are injected using setter methods.

Among these, constructor injection is preferred because it makes dependencies mandatory and avoids null values.”

“Constructor injection is preferred because the required dependencies are provided when the object is created, so the object cannot exist without its dependencies. This makes the code more reliable and avoids null values.”

BeanFactory	ApplicationContext
Basic container in Spring	Advanced container in Spring
Supports basic Dependency Injection	Supports Dependency Injection with extra features
Uses lazy initialization(loading) by default	Uses eager initialization by default
Does not support AOP directly	Supports AOP
Does not support event handling	Supports event handling
Does not support internationalization	Supports internationalization
Less commonly used	Mostly used in Spring applications
Lightweight	More feature-rich

“BeanFactory is the basic container in Spring used for managing beans and dependency injection, whereas ApplicationContext is an advanced container that provides additional features like AOP support, event handling, and internationalization.”

Loose Coupling

“Loose coupling means classes are less dependent on each other. Changes in one class will have minimal impact on another class. It provides flexibility, maintainability, and easier testing.”

Tight Coupling

“Tight coupling means classes are strongly dependent on each other. Changes in one class may affect another class directly, making the application harder to maintain.”

Bean lifecycle refers to the different stages a Spring bean goes through from creation to destruction.

The stages are:

1. Spring container creates the bean
2. Dependencies are injected
3. Initialization method is called
4. Bean is ready for use
5. Destroy method is called before bean removal.”

“Bean scope defines how many objects of a bean are created and how long they are managed by the Spring container.

The main bean scopes in Spring are:

- Singleton
- Prototype
- Request
- Session
- Application

By default, Spring beans use singleton scope, where only one object is created for the entire application.”

Bean scope defines how many bean objects are created and managed by the Spring container. The default scope is singleton, where only one object is created for the entire application.

“Autowiring is a feature in Spring that automatically injects the required dependency objects into a class without creating them manually.”

“@Autowired is an annotation in Spring used to automatically inject dependency objects into a class by the Spring container.”

@Qualifier

“@Qualifier is used to specify which bean should be injected when multiple beans of the same type are available.”

@Primary

“@Primary is used to make a bean the default choice among multiple beans of the same type.”

“@Lazy is an annotation in Spring used to delay bean creation until the bean is actually needed.

By default, Spring creates beans at application startup, but with @Lazy, the bean is created only when it is used for the first time.”

“@Component is a Spring annotation used to mark a class as a Spring bean. During component scanning, the Spring container automatically detects the class, creates its object, and manages it.”

“@Bean is a Spring annotation used to manually create and register an object as a bean inside the Spring container.”

@Component	@Bean
Used on class	Used on method
Spring automatically creates object using component scanning	Developer manually creates object
Used for own application classes	Used for third-party/external classes

<code>@Component</code>	<code>@Bean</code>
Less configuration	More control over bean creation
Automatic bean creation	Manual bean registration

“`@Component` is a generic Spring bean annotation, `@Service` is used for the service layer and business logic, `@Repository` is used for the persistence or database layer, and `@Controller` is used to handle client requests in web applications.”

“Stereotype annotations are special annotations in Spring used to mark classes as Spring beans so that the Spring container can automatically detect and manage them during component scanning.

Examples are:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`”

Component scanning is the process in which the Spring container automatically scans the specified packages, detects classes annotated with stereotype annotations like `@Component`, and creates and manages their objects as Spring beans.”

“Configuration in Spring means providing instructions to the Spring container about how to create, configure, and manage beans in the application.”

`@Configuration` is a Spring annotation used to declare a class as a configuration class that contains bean definitions for the Spring container.”

“XML configuration uses XML files to configure beans and dependencies, whereas annotation configuration uses annotations directly in Java classes. Annotation-based configuration is easier, cleaner, and more commonly used in modern Spring applications.”

“Spring AOP (Aspect-Oriented Programming) is a feature in Spring used to separate cross-cutting concerns like logging, security, and transaction management from the main business logic.”

What is Aspect in AOP?

“An Aspect is a class that contains cross-cutting concerns such as logging, security, or transaction management.”

What is Advice?

“Advice is the action performed by an aspect at a particular point during program execution.”

Types of Advice in Spring AOP

- Before Advice
 - After Advice
 - After Returning Advice
 - After Throwing Advice
 - Around Advice
-

What is JoinPoint?

“JoinPoint is a point during the execution of a program where an aspect can be applied, such as method execution.”

What is Pointcut?

“Pointcut is an expression that specifies at which JoinPoints the advice should be applied.”

What are ACID Properties?

A – Atomicity

Either all operations are completed or none are completed.

C – Consistency

Data should remain valid before and after the transaction.

I – Isolation

One transaction should not affect another transaction.

D – Durability

Once a transaction is committed, the data is permanently saved.”

What is Transaction Management?

“Transaction management is the process of managing database operations as a single unit of work to ensure data consistency and integrity.”

What is `@Transactional`?

“`@Transactional` is a Spring annotation used to manage transactions automatically. If any operation fails, the entire transaction will be rolled back.”

“MVC (Model View Controller) is a design pattern used to separate an application into three layers: Model, View, and Controller.

- Model represents the data and business logic of the application.
- View represents the user interface that displays the data to the user.
- Controller handles the client requests, processes them using the model, and returns the response to the view.

This architecture helps in better code organization, maintainability, and separation of concerns.”

“`DispatcherServlet` is the front controller in Spring MVC that receives all client requests, processes them, and forwards them to the appropriate controller. It acts as the central part of the Spring MVC framework.”

Explain Spring MVC flow?

“First, the client sends a request to the `DispatcherServlet`. The `DispatcherServlet` forwards the request to the appropriate controller. The controller processes the request with the help of the service layer and model. After processing, the controller returns the response to the `DispatcherServlet`, and finally the `DispatcherServlet` sends the response back to the client through the view or as JSON data.”

“Controller is a component in Spring MVC that handles client requests, processes them, and returns the appropriate response.”

“Model is a component in MVC architecture that contains the application data and business-related information. It is used to transfer data between the controller and the view.”

`ViewResolver` helps Spring MVC identify the actual view page from the logical view name returned by the controller by applying prefix and suffix configuration.”

“`@Controller` is used to develop Spring MVC web applications where the controller returns view names that are resolved into actual view pages. `@RestController` is used to develop REST APIs where the response data is returned directly to the client in formats like JSON or XML. `@RestController` internally combines `@Controller` and `@ResponseBody`.”

“`@RequestMapping` is an annotation in Spring MVC used to map HTTP requests to specific controller classes or handler methods.”

GET is used to retrieve data from the server, whereas POST is used to send data to the server for creating or processing resources. In GET, data is passed through the URL, while in POST, data is sent in the request body.”

What is @GetMapping?

“@GetMapping is a Spring MVC annotation used to handle HTTP GET requests, mainly for retrieving data from the server.”

Example:

```
@GetMapping("/employees")
```

What is @PostMapping?

“@PostMapping is used to handle HTTP POST requests, mainly for sending or saving data to the server.”

Example:

```
@PostMapping("/save")
```

What is @PutMapping?

“@PutMapping is used to handle HTTP PUT requests, mainly for updating existing data.”

Example:

```
@PutMapping("/update/{id}")
```

What is @DeleteMapping?

“@DeleteMapping is used to handle HTTP DELETE requests, mainly for deleting data from the server.”

Example:

```
@DeleteMapping("/delete/{id}")
```

@RequestBody is an annotation in Spring used to convert the incoming HTTP request body, usually JSON data, into a Java object automatically.”

“@PathVariable is an annotation in Spring used to extract values from the URL path and bind them to method parameters.”

Example:

```
@GetMapping("/employee/{id}")  
public String getEmployee(@PathVariable int id) {
```

```
}
```

If URL is:

```
/employee/101
```

Then:

```
id = 101
```

“@RequestParam is an annotation in Spring used to retrieve query parameter values from the URL and bind them to method parameters.”

Example:

```
@GetMapping("/employee")
public String getEmployee(@RequestParam int id) {

}
```

URL:

```
/employee?id=101
```

Then:

```
id = 101
```

ResponseEntity is used in Spring to send the complete HTTP response with custom response body, status code, and headers.”

“REST API is a web service that enables communication between client and server using HTTP protocols and standard HTTP methods like GET, POST, PUT, and DELETE.”

HTTP status codes are **three-digit numbers** returned by a web server to indicate the result of a client’s request (usually from a browser). They tell you whether a request succeeded, failed, or needs further action.

HTTP status codes are **3-digit numbers** returned by the server to show the result of a client request.

Common Categories:

- **1xx** → Information
- **2xx** → Success (Example: **200 OK**)
- **3xx** → Redirection (Example: **301 Moved Permanently**)
- **4xx** → Client error (Example: **404 Not Found**)
- **5xx** → Server error (Example: **500 Internal Server Error**)

“PUT is used to update the complete resource, while PATCH is used to update only specific fields of a resource. PUT replaces the entire existing data, whereas PATCH modifies only the required data.”

CORS stands for Cross-Origin Resource Sharing. It is a security mechanism that allows or restricts requests between different origins, such as different domains, ports, or protocols.”

“Spring Boot is a framework built on top of the Spring Framework that is used to develop standalone and production-ready Java applications quickly and easily. It reduces the amount of configuration and boilerplate code by providing features like auto-configuration, embedded servers, and starter dependencies. It is mainly used for developing REST APIs, microservices, and web applications.”

Advantages of Spring Boot

- Reduces configuration work through auto-configuration
- Provides embedded servers like Tomcat, so no need for external server setup
- Faster application development with less boilerplate code
- Easy dependency management using starter dependencies
- Simplifies creation of REST APIs and microservices
- Provides production-ready features like logging and monitoring
- Easy to deploy and run as standalone applications

“Spring Framework provides the core features for developing Java applications, but it requires more configuration and manual setup. Spring Boot is built on top of Spring Framework and simplifies development by providing auto-configuration, embedded servers, and starter dependencies, which helps in developing applications faster with less configuration.”

“@SpringBootApplication is the main annotation in Spring Boot used to mark the main class of the application. It is a combination of three annotations: @Configuration, @EnableAutoConfiguration, and @ComponentScan. It enables auto-configuration, component scanning, and Java-based configuration in a Spring Boot application.”

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

What is application.properties?

“application.properties is a configuration file in Spring Boot used to store application settings such as database configuration, server port, and other properties.”

Example:

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/test
```

What is Spring Initializr?

“Spring Initializr is a tool used to create and bootstrap Spring Boot projects with required dependencies and project structure.”

Used to generate:

- Maven/Gradle project
- Dependencies
- Basic project setup

What is Embedded Server?

“An embedded server is a server that comes built into the Spring Boot application, so there is no need to install and configure an external server manually.”

Examples:

- Tomcat
- Jetty

Auto Configuration

“Auto-configuration means Spring Boot automatically sets up required configurations based on the dependencies added in the project.”

Starter Dependency

“Starter dependency is a single dependency that contains all required related dependencies.”

What is Auto Configuration?

“Auto-configuration is a feature in Spring Boot that automatically configures the application based on the dependencies present in the project.”

Example:

If MySQL dependency is added, Spring Boot automatically configures database settings.

What is Starter Dependency?

“Starter dependencies are pre-defined dependency packages in Spring Boot that simplify dependency management.”

Example:

spring-boot-starter-web

It automatically includes:

- Spring MVC
- Tomcat
- JSON libraries

Difference between application.properties and application.yml?

- [Interview Style Answer](#)
- “Both application.properties and application.yml are configuration files in Spring Boot used to store application settings. The main difference is in their format. application.properties stores data in key-value format, whereas application.yml stores data in hierarchical YAML format, which is more readable for complex configurations.”

- **Example**

- [application.properties](#)

- server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/test

- [application.yml](#)

- server:
 port: 8080

 spring:
 datasource:
 url: jdbc:mysql://localhost:3306/test

[What is Actuator?](#)

“Spring Boot Actuator is used to monitor and manage the application. It provides production-ready features like health check, metrics, and application information.”

Example endpoints:

- /health
- /metrics

[What is DevTools?](#)

“Spring Boot DevTools is a development tool that improves developer productivity by providing features like automatic restart and live reload during development.”

[How does Spring Boot reduce boilerplate code?](#)

“Spring Boot reduces boilerplate code by providing auto-configuration, starter dependencies, and embedded servers, which minimizes manual configuration and setup.”

What happens when Spring Boot application starts?

“When a Spring Boot application starts, the Spring container is created, beans are initialized, auto-configuration is applied, embedded server starts, and the application becomes ready to handle requests.”

Spring Data JPA & Hibernate

What is JPA?

“JPA (Java Persistence API) is a specification used to manage database operations in Java applications.”

What is Hibernate?

“Hibernate is an ORM framework and implementation of JPA used to interact with databases.”

Difference between JPA and Hibernate?

“JPA is a specification, whereas Hibernate is an implementation of JPA.”

What is ORM?

“ORM (Object Relational Mapping) is a technique used to map Java objects to database tables.”

What is Entity?

“An Entity is a Java class that represents a database table.”

What is `@Entity`?

“`@Entity` is an annotation used to mark a class as a JPA entity.”

```
@Entity
public class Employee {
```

```
}
```

What is @Id?

“@Id is used to specify the primary key of an entity.”

```
@Id  
private int id;
```

What is @GeneratedValue?

“@GeneratedValue is used to automatically generate primary key values.”

```
@GeneratedValue  
private int id;
```

What is @Table?

“@Table is used to specify the database table name for an entity.”

```
@Table (name="employee")
```

What is @Column?

“@Column is used to map a class field to a database column.”

```
@Column (name="emp_name")  
private String name;
```

What is JpaRepository?

“JpaRepository is an interface provided by Spring Data JPA used to perform database operations like save, update, delete, and fetch data.”

Difference between CrudRepository and JpaRepository?

“CrudRepository provides basic CRUD operations, whereas JpaRepository extends CrudRepository and provides additional JPA-related features like pagination and sorting.”

Common methods in JpaRepository?

- save() → save or update data
- findById() → fetch data by id

- `findAll()` → fetch all records
 - `deleteById()` → delete record by id
 - `existsById()` → checks record exists or not
-

What is FetchType?

“FetchType defines when related entity data should be loaded from the database.”

Types:

- LAZY
 - EAGER
-

Difference between LAZY and EAGER loading?

- **LAZY** → Related data loads only when needed
 - **EAGER** → Related data loads immediately along with parent entity
-

What is Cascade?

“Cascade is used to automatically apply operations performed on parent entity to child entity.”

Types of Cascade?

- PERSIST
 - MERGE
 - REMOVE
 - REFRESH
 - DETACH
 - ALL
-

What is mappedBy?

“mappedBy is used to specify the owner side of a bidirectional relationship.”

Explain OneToOne relationship?

“One record is related to only one record.”

Example:
One person → One passport

Explain OneToMany relationship?

“One record is related to multiple records.”

Example:
One department → Many employees

Explain ManyToOne relationship?

“Many records are related to one record.”

Example:
Many employees → One department

Explain ManyToMany relationship?

“Many records are related to many records.”

Example:
Many students → Many courses

What is Native Query?

“Native query is an SQL query written directly using database-specific SQL syntax.”

Example:

```
SELECT * FROM employee;
```

What is JPQL?

“JPQL (Java Persistence Query Language) is an object-oriented query language used to query entity objects instead of database tables.”

Example:

```
SELECT e FROM Employee e
```

Difference between `save()` and `saveAndFlush()`?

- **`save()`** → Saves data, but changes may be stored in database later
 - **`saveAndFlush()`** → Saves data and immediately flushes changes to database
-

Interview Style Answer

“`save()` stores data temporarily in persistence context and commits later, whereas `saveAndFlush()` immediately synchronizes changes with the database.”

What is Spring Security?

“Spring Security is a framework used to provide authentication and authorization for Java and Spring Boot applications.”

Authentication vs Authorization?

- **Authentication** → Verifying who the user is
- **Authorization** → Verifying what the user can access

Example:

- Login → Authentication
 - Access admin page → Authorization
-

What is JWT?

“JWT (JSON Web Token) is a token-based authentication mechanism used to securely transfer user information between client and server.”

Structure of JWT?

JWT contains 3 parts:

1. Header
2. Payload
3. Signature

Format:

`header.payload.signature`

Why JWT is used?

- Secure authentication
 - Stateless authentication
 - No need to store session on server
 - Mostly used in REST APIs
-

What is BCryptPasswordEncoder?

“BCryptPasswordEncoder is used to encrypt passwords before storing them in the database.”

What is Stateless Authentication?

“In stateless authentication, server does not store session data. Each request contains authentication information like JWT token.”

What is Role-Based Authorization?

“Role-based authorization restricts access to resources based on user roles like ADMIN or USER.”

Explain JWT Flow?

1. User sends username and password
2. Server validates credentials
3. Server generates JWT token
4. Token is sent to client
5. Client sends token in every request
6. Server validates token and gives response

Example:

Authorization: Bearer jwt-token